

NoSQL

FERCOQ FLORENT

Historique

Histoire des bases de données

La taille, les fonctionnalités et les performances des bases de données et des SGBD ont crû de manière exponentielle au cours du temps

Ces augmentations de performances ont été rendues possibles par les progrès technologiques des processeurs, mémoires, stockages de masse et réseaux de communication

L'histoire des bases de données a connu 3 phases :

- Navigationnelle
- Relationnelle (SQL)
- Post-relationnelle

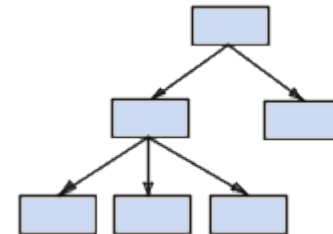
Bases de données navigationnelles

Le terme "bases de données" apparaît avec les dispositifs de stockage à accès direct (rouleaux / disques) à partir des années 60

Cela représentait une avancée, par rapport aux systèmes à bandes existants, qui permit un usage interactif partagé plutôt que des traitements "batchs" quotidiens

Les premières bases de données sont apparues selon le modèle hiérarchique.

- Informations découpées en deux niveaux de hiérarchie
- Puis sur la forme d'un diagramme en arbre



L'intérêt pour un standard commence à apparaître, Charles Bachman, auteur d'IDMS, fonda le *Data Base Task Group* avec le groupe CODASYL.

Bases de données navigationnelles

En 1971, le *Data Base Task Group* sort son standard : l'approche CODASYL

Celle-ci se base sur une navigation “manuelle” de données liées entre elles dans un réseau. Les applications peuvent trouver des données :

- Via un clé primaire
- Par navigation d’une donnée à une autre
- En parcours toutes les données dans l’ordre
- Plus tard, via des arbres "B-Tree"

Bases de données relationnelles

Les bases CODASYL avaient un langage de requêtes très simple

Au final, elles étaient complexes et demandaient beaucoup d'effort et de connaissances pour produire des applications utiles

Edgar Codd (IBM), insatisfait de l'approche CODASYL, en particulier par l'absence de fonctionnalités de recherche, écrit en 1970 plusieurs papiers qui présentent une nouvelle approche pour stocker et travailler avec de larges ensembles de données

Plutôt que de stocker des données à format libre sous forme de listes liées, il propose d'utiliser des "tables" de données à format fixé, chaque table supportant une entité différente.

Bases de données relationnelles

Le modèle relationnel permet de mieux gérer des données hétérogènes, et pas toujours présentes (listes avec vides => n tables pouvant être liées)

Le modèle relationnel permet plus facilement l'évolution de la base de données, par des entités référençant d'autres entités.

Trois modèles permettent de répondre aux besoins de modèles de données des applications :

- relations *one-to-many* comme le modèle hiérarchique
- relations *many-to-many* comme le modèle réseau
- données tabulaires

Bases de données relationnelles

Lier les données est la clé des bases relationnelles, une clé (primaire) identifie une donnée de manière unique

Dans une autre entité de données, une autre clé (étrangère) permet de référencer la clé (primaire) d'une autre entité.

Plus tard est apparu le langage SQL pour formaliser et normaliser l'interrogation de ces bases de données :

- 1986 SQL-86/SQL-87 Première formalisation par l'ANSI
- 1989 SQL-89 Révision mineure
- 1992 SQL-92/SQL2 Révision majeure (ISO 9075)
- 1999 SQL:1999/SQL3 Expressions régulières, requêtes récursives, triggers, ...
- 2003 SQL:2003 Fonctionnalités XML, fonctions de fenêtrage, séquences...
- 2006 SQL:2006 Ajout de fonctionnalités XML...
- 2008 SQL:2008 INSTEAD OF, TRUNCATE, FETCH, ...
- 2011 SQL:2011 PERIOD FOR...
- 2016 SQL:2016 JSON...

Des nouveaux besoins

Dans les années 90, en parallèle avec la montée en popularité des langages de programmation objet, est apparu un nouveau modèle de base de données : le modèle objet

- Cela permet des relations avec des objets et non plus des champs individuels

Cela évite également les traductions entre un langage objet et une base de données relationnelle

Suivant l'apparition du XML, sont apparues également les bases de données XML natives

- Elles permettent également un stockage direct de ce type d'informations et l'utilisation des langages de requêtes liés à ce langage

L'omniprésence du modèle relationnel

Fin des années 90, les bases relationnelles sont les plus répandues :

- Modèle de données simple et normalisé : relations, tuples, attributs, clés primaires et étrangères,...
- Modèle de requêtes s'appuyant sur l'algèbre relationnelle
- Requêtes normalisées en SQL
- Indépendance par rapport au stockage physique
- Garantie « ACID » (Atomicité, Cohérence, Isolation, Durabilité)

Ces différents points ont permis aux bases relationnelles de s'imposer par rapport aux modèles « hiérarchique » et « réseau »

Les premiers SGBDR

En 1974, IBM a commencé à développer System R, un projet de recherche visant à développer un prototype de SGBDR

System R était un projet fondateur: il s'agissait de la première implémentation de SQL, qui est depuis devenu le langage de requête de données relationnel standard

C'était également le premier système à démontrer qu'un SGBDR pouvait fournir de bonnes performances de traitement des transactions

Ces décisions de conception et certains choix d'algorithmes fondamentaux ont influencé de nombreux systèmes relationnels ultérieurs

Les premiers SGBDR

Le premier SGBDR disponible dans le commerce était Oracle, publié en 1979 par Relational Software, renommé depuis Oracle Corporation

En 1981, Informix Corporation a publié son propre SGBDR sous le nom Informix

En 1981, Le premier produit commercial d'IBM, SQL/DS, est disponible pour les systèmes d'exploitation DOS/VSE et VM/CMS (mainframes).

- Le nom DB2 a été donné à ce SGBDR en 1983 quand IBM l'a publié sur MVS
- Par la suite, IBM a porté DB2 à d'autres plates-formes : OS/2, UNIX et Windows, puis Linux et les PDA

Les premiers SGBDR

En 1987, Sybase Corporation a commercialisé le SGBDR Sybase SQL Server à l'origine pour les plateformes Unix

En 1988, SQL Server pour OS/2 a été développé conjointement pour le PC par Sybase, Microsoft et Ashton-Tate

- Microsoft restera le seul partenaire après le portage de SQL Server sous Windows NT

En 1993, l'accord de licence de co-développement conclu entre Microsoft et Sybase a pris fin

- Les sociétés se sont séparées tout en poursuivant le développement de leurs versions respectives de SQL Server
- Actuellement la version de Sybase est plus connue sous le nom Sybase ASE

Les principaux acteurs SGBDR

Selon DB-Engines (<https://db-engines.com/>), en mai 2024, les SGBDR les plus populaires étaient :

- Oracle
- MySQL
- Microsoft SQL Server
- PostgreSQL
- IBM DB2
- SQLite
- Microsoft Access

Le Big Data

Big Data – Une accélération

2022

Source :

<https://www.domo.com/data-never-sleeps>



Big Data

Le big data désigne des ensembles de données devenus si volumineux qu'ils dépassent les capacités des outils informatiques classiques de gestion de base de données

L'explosion quantitative (et souvent redondante) de la donnée numérique contraint à de nouvelles manières de voir et analyser le monde

De nouveaux ordres de grandeur concernent la capture, le stockage, la recherche, le partage, l'analyse et la visualisation des données

Les perspectives du traitement des big data sont vastes et diversifiées

On commence à voir des IA explorer le big data et apprendre automatiquement via des réseaux de neurones artificiels par exemple

Big Data

Au-delà de la quantité de données, le big data s'accompagne d'une variété de données : brutes, semi-structurées, non structurées

La démultiplication des outils de collecte sur les individus et sur les objets permet d'amasser toujours plus de données

Et les analyses sont d'autant plus complexes qu'elles portent de plus en plus sur les liens entre des données de natures différentes

Des flux croissants de données doivent être analysés en quasi-temps réel pour répondre aux besoins des processus chrono-sensibles (ex: trading haute-fréquence).

Les SGBDR classiques ne suffisent plus forcément à gérer ces problématiques

Big Data

Données structurées

- Ce sont des informations qui s'intègrent dans modèle de données bien établi
- On retrouve les données dans des champs définis
- C'est typiquement le genre de données géré par les SGBDR et interrogé via le SQL

Données non-structurées

- Il s'agit de données présentes sous forme brute
- Cela peut être un texte, un email, une conversation, un article, une image
- Elle nécessite d'être analysée et traitée

Données semi-structurées

- Il s'agit de données partiellement structurées
- On garde une certaine structure mais cette dernière n'est pas rigide

Les bases de données NoSQL

Pourquoi le NoSQL ?

Il y a 5 ans environ : Amazon a trouvé que 0,1s de latence lui coûtait 1% de perte de chiffre d'affaires

- Pour mémoire, le CA du T1 2018 s'élevait à 51 Md de \$

Google calculait que $\frac{1}{2}$ s d'augmentation du temps de réponse de son moteur de recherches, lui faisait perdre 20% de trafic.

- Pour mémoire, le nombre mensuel de requêtes de Google en 2018 s'élevait à 100 Mds
- CA 2017 : 110 Mds de \$ et bénéfice 2017 : 26 Mds de \$

Emergence du mouvement NoSQL

Avant l'avènement du big data, les bases relationnelles sont les plus répandues :

- Modèle de données simple et normalisé : relations, tuples, attributs, clés primaires et étrangères, ...
- Modèle de requêtes s'appuyant sur l'algèbre relationnelle
- Requêtes normalisées en SQL
- Indépendance par rapport au stockage physique
- Garantie « ACID » (Atomicité, Cohérence, Isolation, Durabilité)

Ces différents points ont permis aux bases relationnelles de s'imposer par rapport aux modèles « hiérarchique » et « réseau »

Emergence du mouvement NoSQL

Les grandes entreprises du web ont été confrontées aux limitations des SGBD relationnels en termes de scalabilité

Elles ont développé leurs propres systèmes de bases de données (Google : BigTable, Facebook : Cassandra, ...)

Fin des années 2000, apparition de plusieurs projets open source pour répondre à ces besoins sous différentes formes : clefs / valeurs, orientés documents, orientés colonnes, orientés graphes.

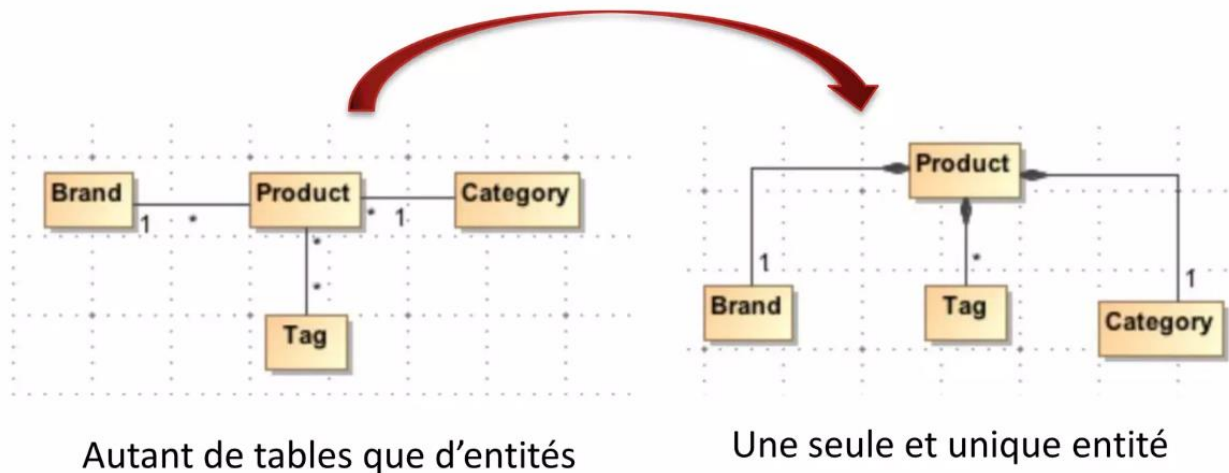
En 2009, le terme « NoSQL » apparaît comme intitulé d'une rencontre d'informaticiens à San Francisco.

Les modèles de données NoSQL

Les données sont agrégées

L'agrégation obéit en général au modèle de consultation

Toutes les entités censées être restituées ensemble sont agrégées au sein d'une même structure



Les SGBD NoSQL

NoSQL

Le terme « NoSQL » désigne un type de base de données qui n'utilise pas le modèle relationnel.

Généralement, le langage SQL n'est pas utilisé pour requêter ces bases de données

Le terme NoSQL est trompeur car il signifie *Not Only SQL* (pas seulement SQL). On aurait plutôt dû l'appeler *Not Only Relational Databases*

NoSQL est une mouvance, et non une technologie spécifique ou un produit

Il y a plusieurs centaines de SGBD NoSQL recensés

- Voir <https://hostingdata.co.uk/nosql-database/>

Des acteurs comme Google, LinkedIn, Amazon ou Facebook conçoivent et exploitent des bases de données de type NoSQL

NoSQL

Quelques systèmes de gestion de base de données NoSQL



IBM Cloudant



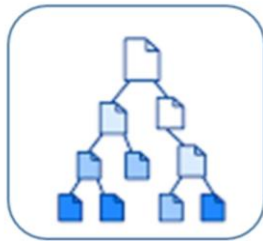
Les SGBD NoSQL Open Source

Bases orientées "clé/valeur" : Redis, Riak, etc

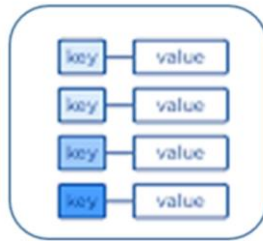
Bases orientées "colonne" : Cassandra, HBase, etc

Bases orientées "document" : MongoDB, Couchbase, ElasticSearch, etc

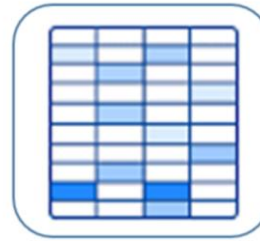
Bases orientées "graphe" : Neo4J, etc



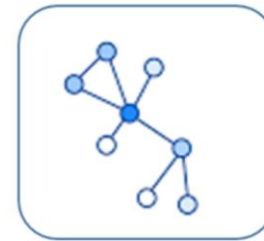
Document
Store



Key-Value
Store



Wide-Column
Store



Graph
Store

Les offres purement Cloud : Azure SQL, Amazon, Google, Atlas

NoSQL

Dans une base de données non relationnelle :

- Pas de notion de schéma : au sein d'une même « table », les « enregistrements » peuvent avoir une structure totalement différente
- Les champs peuvent être ajoutés, modifiés, supprimés d'un enregistrement à l'autre et ce à n'importe quel moment
- Pas de champs inutilisés
- Pas de types de données (ils sont implicites)
- La plupart des traitements sont faits dans la couche application

NoSQL

Pourquoi utiliser NoSQL ?

Parfait pour les bases de données larges et distribuées

Besoin de stocker des données

- Non structurées (à la différence de données insérées dans une table qui doivent correspondre à un schéma bien précis : les différents champs de la table)
- Dont le nombre / type de champs peut varier d'un enregistrement à l'autre
- Exemples : logs, données scientifiques, pages web,...

Facilité de mise en place et de développement

Scalabilité horizontale aisée (= ajout de serveur)

NoSQL

Particularités

NoSQL supprime toute la complexité des requêtes SQL.

NoSQL fonctionne très bien en cluster.

NoSQL peut gérer un nombre très important de données (ce qu'on appelle le Big Data)

Les SGBD NoSQL sont en grande partie open source

Dans la grande majorité des bases de données NoSQL, aucune gestion des transactions n'est prévue

- Elles doivent être gérées au niveau de la couche applicative

Bases orientées *clé/valeur*

Bases orientées *clé/valeur*

Types de base de données NoSQL

Clé	Valeur
Eric	+33 6 1234657
Franck	0610101011
Mélissa	Paris, France
Paul	0610101012, Paris, France
David	1.80m, Nantes
Raoul	+33 6 2345678, Reims

Permet d'associer une valeur à une clé
sous la forme d'un tableau associatif

Bases orientées *clé/valeur*

Une base de données clés-valeurs est un paradigme de stockage de données conçu pour stocker, récupérer et gérer des tableaux associatifs, également appelée dictionnaire ou table de hachage

Les dictionnaires contiennent une collection d'objets, ou enregistrements, qui peuvent contenir à leur tour des champs différents, chacun contenant des données

Ces enregistrements sont stockés et récupérés à l'aide d'une clé qui identifie de manière unique l'enregistrement et est utilisée pour rechercher rapidement les données dans la base de données

Bases orientées *clé/valeur*

Fonctionnent de manière très différente des bases de données relationnelles

- Elles traitent les données comme une seule collection opaque, qui peut avoir des champs différents pour chaque enregistrement

Ceci offre une flexibilité considérable et s'approche des concepts tels que la programmation orientée objet

Comme les valeurs facultatives ne sont pas représentées par des espaces réservés, comme dans la plupart des bases de données relationnelles, les bases de données clé-valeur utilisent souvent moins de mémoire pour stocker une base de données équivalente, ce qui peut entraîner des gains de performances dans certains cas d'usages

Bases orientées *clé/valeur*

Les bases de données clé-valeur peuvent utiliser des modèles de cohérence allant de la cohérence éventuelle à la sérialisabilité

Certains supportent le tri de clés

Certains conservent des données en mémoire, d'autres utilisent des disques SSD ou "durs"

Elles fournissent des opérations généralement simples pour manipuler les données : création, lecture, modification, suppression

La simplicité fait que toute la logique de gestion des données est située dans l'application client

Souvent utilisé comme solution de cache ou des tables de hashages

Bases orientées *clé/valeur*

Le marché des bases orientées clé/valeur est très fourni, avec beaucoup de solutions open-source

- Redis (<https://redis.io/>)
- Couchbase (<https://www.couchbase.com/>) - Supporte entre autres le modèle clé/valeur
- ArangoDB (<https://arangodb.com/>) - Supporte entre autres le modèle clé/valeur
- Memcached (<https://memcached.org/>)
- Amazon DynamoDB (<https://aws.amazon.com/fr/dynamodb/>)

Beaucoup de bases orientées clé/valeur sont utilisées afin d'améliorer les temps d'accès à des sites ou applications web



Redis

UN EXEMPLE DE BASE ORIENTÉE CLÉ/VALEUR

Redis

Redis (<https://redis.io/>) est un SGDB NoSQL de type clé/valeur

- Le nom provient de *Remote Dictionary Server*

Le projet a vu le jour en 2009 puis a fortement évolué au fil des ans

Aujourd'hui, c'est le 6^{ème} SGBD le plus utilisé selon <https://db-engines.com/en/>

Redis permet de manipuler des données simples et de les stocker en RAM

- Ce stockage en RAM assure une haute performance de la part de Redis
- Pour assurer la sauvegarde des données, on peut demander à Redis de faire des captures de la base dans un fichier

Installation

Pour utiliser Redis, on dispose de plusieurs solutions

- L'installation native
 - Sur Linux, cela consiste en l'installation des paquets adéquats - https://redis.io/docs/latest/operate/oss_and_stack/install/install-redis/install-redis-on-linux/
 - Sur MacOS, cela consiste en l'installation de paquets via brew - https://redis.io/docs/latest/operate/oss_and_stack/install/install-stack/mac-os/
 - Sur Windows, Redis n'est pas supporté. Il convient dans ce cas d'utiliser WSL
 - Suite à l'installation, on dispose de 2 programmes :
 - `redis-server` pour lancer le serveur redis
 - `redis-cli` pour lancer le client redis
- L'utilisation d'une image Docker
 - Une image officielle est disponible : `redis` - https://hub.docker.com/_/redis
 - Une petit `docker run` et c'est parti !
- L'utilisation de Redis en mode SaaS
 - Via le lien *Try Redis* sur <https://redis.io/>

Utilisation

Une fois la base de données Redis créée, on peut s'y connecter avec l'utilitaire `redis-cli`

- Seul problème, `redis-cli` n'existe pas nativement sur Windows !

Solutions pour Windows :

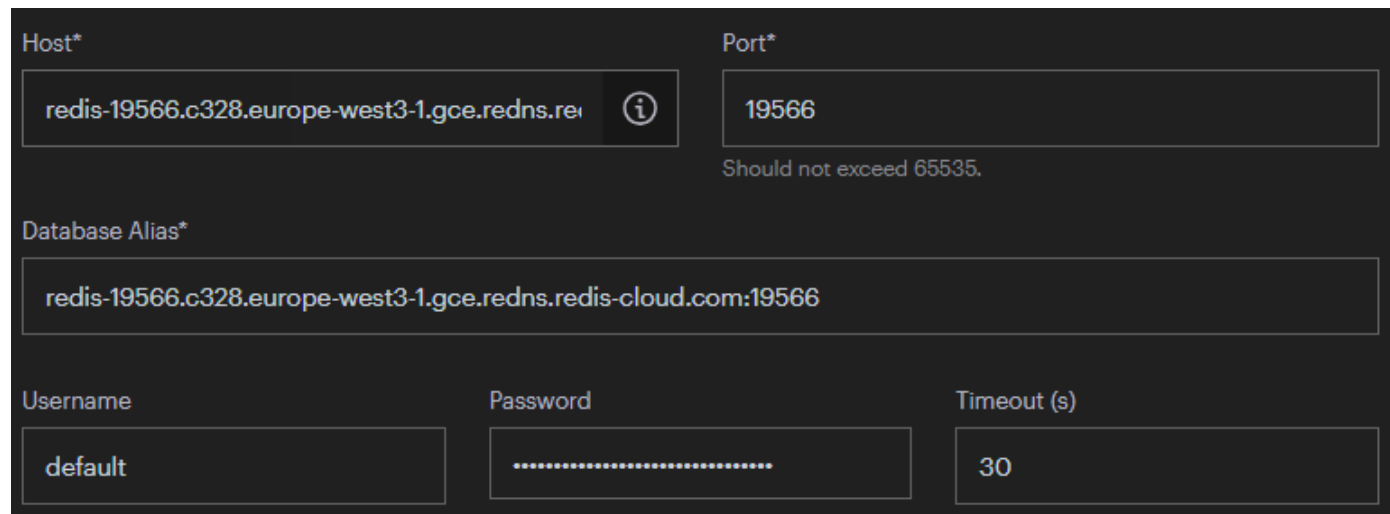
- Utiliser la version Node.js de `redis-cli` (<https://www.npmjs.com/package/redis-cli>) qui permet d'avoir accès à une commande `rdcli`
- Utiliser l'outil graphique *RedisInsight* (<https://redis.io/insight/>)

Utilisation

Redis en mode SaaS nous permet de voir la commande pour se connecter à notre base Redis

- `redis-cli -u redis://default:*****@redis-19566.c328.europe-west3-1.gce.redns.redis-cloud.com:19566`

Dans *RedisInsight*, on peut déclarer une nouvelle connexion et remplir le formulaire



The screenshot shows the RedisInsight connection configuration interface. It features a dark theme with light-colored text and input fields. The form is organized into several sections: Host*, Port*, Database Alias*, Username, Password, and Timeout (s). The Host* field contains 'redis-19566.c328.europe-west3-1.gce.redns.rei' and has an information icon. The Port* field contains '19566' and a note 'Should not exceed 65535.'. The Database Alias* field contains 'redis-19566.c328.europe-west3-1.gce.redns.redis-cloud.com:19566'. The Username field contains 'default'. The Password field is masked with dots. The Timeout (s) field contains '30'.

Host*	Port*	Database Alias*	Username	Password	Timeout (s)
redis-19566.c328.europe-west3-1.gce.redns.rei ⓘ	19566 Should not exceed 65535.	redis-19566.c328.europe-west3-1.gce.redns.redis-cloud.com:19566	default		30

Redis avec Docker

Pour avoir à disposition un serveur Redis, on peut aussi utiliser l'image officielle Docker appelée `redis`

- https://hub.docker.com/_/redis

La commande de base pour lancer une instance Redis

- `docker run -d --name serveur-redis redis`

Par défaut, c'est le port 6379 qui est exposé par le serveur redis. On peut donc faire un *mapping* de port

- `docker run -d --rm --name redis -p 6379:6379 redis`

Ou bien, comme mentionné dans la documentation, on peut avoir à disposition un client redis en se mettant sur le même réseau *bridge*

- `docker network create some-network`
- `docker run -it --network some-network --rm redis redis-cli -h some-redis`
- Par défaut `redis-cli` se connecte au serveur à l'adresse `127.0.0.1` sur le port 6379. On peut changer l'hôte Redis (*hostname* ou IP) avec l'option `-h` et le port avec l'option `-p`

Commandes génériques

Ensuite, Redis va fonctionner grâce à l'utilisation de commandes que l'on va pouvoir saisir

Quelques commandes génériques :

- `PING` pour tester une connexion, la commande doit retourner `PONG`
- `CONFIG GET *` pour obtenir la configuration du serveur

Gestion des clés

Commande SET

- Permet d'ajouter une clé dans la base
 - `SET ville-01 Paris` retourne OK
- On peut utiliser l'option `NX` pour éviter d'écraser une clé déjà existante (`SET ville-01 Paris NX`) ou `XX` pour mettre à jour uniquement une clé déjà existante (`SET ville-01 Paris XX`)
- On peut utiliser `MSET` pour déverser plusieurs clés/valeurs d'un coup (`MSET cle1 val1 cle2 val2 cle3 val3...`)

Commande DEL

- Permet de supprimer une clé dans la base et retourne le nombre de clés supprimées (on peut après `DEL` indiquer plusieurs clés)
 - `DEL ville-01` retourne 0 ou 1

Commande EXISTS

- Pour savoir si une ou plusieurs clés existent (retourne le nombre de clés qui existent)
 - `EXISTS ville-01` retourne 0 ou 1

Gestion des clés

Commande KEYS

- Pour rechercher des clés qui correspondent un modèle
 - `KEYS ville-*` retourne le nom des clés qui correspondent

Commande RENAME

- Permet de renommer une clé (si la nouvelle clé existe déjà, elle est écrasée)
 - `RENAME ville-03 ville-06` renomme la clé `ville-03` en `ville-06` et retourne OK
 - On peut utiliser `RENAMENX` pour éviter d'écraser une clé existante

Commande TYPE

- Retourne le type de donnée associé à une clé
 - `TYPE ville-01` retourne `string`

Gestion des clés

Pour organiser sa données, on utilisera des clés sous la forme
xxx:yyy:zzz

Exemple :

- SET groupes:red-hot-chili-peppers:basse Flea
- SET groupes:red-hot-chili-peppers:batterie 'Chad Smith'
- SET groupes:red-hot-chili-peppers:chant 'Anthony Kiedis'
- SET groupes:maroon-5:chant 'Adam Levine'

✓	groupes	2%	4
✓	maroon-5	<1%	1
STRING	chant	No limit	80 B
✓	red-hot-chili-peppers	2%	3
STRING	basse	No limit	88 B
STRING	batterie	No limit	104 B
STRING	chant	No limit	104 B

Dans *Redis Insight* on se retrouve avec une structure représentée sous la forme de dossiers

Gestion des données

On a déjà vu SET pour ajouter une clé dans la base

Commande GET

- Permet de récupérer la valeur d'une clé
 - `GET ville-01` retourne `Paris` ou `(nil)` si la clé n'existe pas

Commande MGET

- Permet de récupérer la valeur de plusieurs clés
 - `MGET ville-01 ville-02 ville-03` retourne plusieurs résultats avec `null` si une clé n'existe pas

Commande GETSET

- Permet de récupérer la valeur d'une clé puis de la mettre à jour
 - `GETSET ville-01 Bruxelles` retourne `Paris` mais change la valeur à `Bruxelles`

Pour la liste des commandes liées aux chaînes de caractères :

<https://redis.io/docs/latest/commands/?group=string>

Gestion des données

Pour la gestion des données de type entier, on dispose de quelques commandes

- SET compteur 0

Commande	Utilité	Exemple
INCR <clé>	Incrémente la valeur de 1 et retourne la nouvelle valeur	INCR compteur
INCRBY <clé> <n>	Incrémente la valeur de n et retourne la nouvelle valeur	INCRBY compteur 5
DECR <clé>	Décrémente la valeur de 1 et retourne la nouvelle valeur	DECR compteur
DECRBY <clé> <n>	Décrémente la valeur de n et retourne la nouvelle valeur	DECRBY compteur 5

Pour les types flottants, on dispose aussi de INCRBYFLOAT pour ajouter une valeur à un flottant (on indiquera un nombre négatif pour enlever une valeur)

Les hash

Un *hash* constitue une association de clés / valeurs, un peu comme un objet JavaScript

- Il n'est pas possible de créer un *hash* dans un *hash*

Commande `HSET <cle> <champ> <valeur> [<champ> <valeur> ...]`

- Permet d'ajouter des champs dans un *hash* ou de le créer
 - `HSET albums:maroon-5:1 nom 'Songs about Jane' annee 2002`

Commande `HDEL <cle> <champ> [<champ> ...]`

- Pour supprimer un ou plusieurs champs dans un *hash*
 - `HDEL albums:maroon-5:1 annee`

Commande `HGET <cle> <champ>`

- Pour obtenir la valeur d'un champs dans un *hash*
 - `HGET albums:maroon-5:1 nom`

Commande `HGETALL <cle>`

- Pour obtenir le noms des champs et leurs valeurs dans un *hash*
 - `HGETALL albums:maroon-5:1`

Les hash

Commande `HKEYS <cle>`

- Pour obtenir les champs d'un *hash*
 - `HKEYS albums:maroon-5:1`

Commande `HVALS <cle>`

- Pour obtenir les valeurs d'un *hash*
 - `HVALS albums:maroon-5:1`

Commande `HMGET <cle> <champ> [<champ> ...]`

- Permet d'obtenir les valeurs de plusieurs champs dans un *hash*
 - `HMGET albums:maroon-5:1 nom annee`

Pour la liste des commandes liées aux *hash* :

<https://redis.io/docs/latest/commands/?group=hash>

Les listes

Les listes (*list*) constituent une liste de chaînes de caractères, triés selon leur ordre d'insertion

Commandes LPUSH/RPUSH <cle> <val> [<val> ...]

- Ajoute une ou plusieurs valeurs en début/fin de liste
- LPUSH courses tomates pommes chips

Commande LPOP/RPOP <cle>

- Enlève et retourne le premier/dernier élément d'une liste
- RPOP courses

Commande LLEN <cle>

- Obtenir la taille d'une liste
- LLEN courses

Pour la liste des commandes liées aux listes :

<https://redis.io/docs/latest/commands/?group=list>

Les sets

Les *sets* constituent une collection non-ordonnée de chaînes de caractères

Commande SADD <cle> <elt> [<elt> ...]

- Ajoute un ou plusieurs éléments dans un *set*
 - SADD favoris 'Genesis' 'Red Hot Chili Peppers' 'Jamiroquai'

Commande SCARD <cle>

- Obtenir le nombre d'éléments dans un *set*
 - SCARD favoris

Commande SREM <cle> <elt> [<elt> ...]

- Supprime un ensemble d'éléments d'un *set*
 - SREM favoris 'Jamiroquai'

Commande SPOP <cle>

- Supprime et retourne un élément aléatoire d'un *set*
 - SPOP favoris

Commande SMOVE <src> <dest> <cle>

- Déplace un élément aléatoire d'un *set* vers un autre *set*
 - SMOVE favoris oublies 'Genesis'

Pour la liste des commandes liées aux *sets* : <https://redis.io/docs/latest/commands/?group=set>

Les sorted sets

Les *sorted sets* constituent une collection ordonnée d'éléments

- Ils n'acceptent pas les doublons, contrairement aux listes
- A chaque élément est associé un score qui vient déterminer son classement

Commande `ZADD <cle> <score> <elt> [<score> <elt> ...]`

- Permet d'ajouter des éléments dans un *sorted set*
 - `ZADD favoris 5 'Genesis' 1 'Red Hot Chili Peppers'`

Commande `ZCARD <cle>`

- Obtenir le nombre d'éléments dans un *sorted set*
 - `ZCARD favoris`

Commande `ZCOUNT <cle> <min> <max>`

- Retourne le nombre d'éléments dont le score est dans l'intervalle indiqué
 - `ZCOUNT favoris 2 10`

Les sorted sets

Commande `ZRANGE <cle> <min> <max> [WITHSCORES]`

- Retourne un classement d'éléments selon leur score (commence à 0)
 - `ZRANGE favoris 0 2` pour avoir le top 3

Commande `ZRANK <cle> <element>`

- Obtenir le classement d'un élément (commence à 0)
 - `ZRANK favoris Genesis`

Commande `ZSCORE <cle> <element>`

- Obtenir le score d'un élément
 - `ZSCORE favoris Genesis`

Pour la liste des commandes liées aux *sorted sets* :

<https://redis.io/docs/latest/commands/?group=sorted-set>

Tout vider

Pour vider toute la base, on dispose de la commande `FLUSHALL`

La purge de la base peut se faire de façon synchrone ou asynchrone

Pour sélectionner le mode, on dispose de deux mots-clés :

- `SYNC` : synchrone (par défaut)
- `ASYNC` : asynchrone

Exemple : `FLUSHALL ASYNC`

Les transactions

Les transactions dans Redis permettent d'exécuter un groupe de commande en une seule étape

On lance une transaction avec la commande `MULTI`

On vient ensuite saisir toutes les commandes souhaitées

On exécute les commandes de la transaction avec la commande `EXEC`

Pour annuler une transaction, c'est la commande `DISCARD` qui sera utilisée

Autres possibilités

Redis offre beaucoup d'autres possibilités en terme de fonctionnalités, par exemple :

- Gérer une expiration des clés
 - Commande `EXPIRE <cle> <secondes>` pour oublier une clé au bout d'un certain temps (PEXPIRE en ms) Commande `EXPIREAT <cle> <timestamp>` pour oublier une clé à un moment particulier (*timestamp*)
 - PEXPIREAT pour indiquer un timestamp en millisecondes
 - Commande `PERSIST <cle>` pour enlever l'expiration d'une clé
 - Commande `TTL <cle>` pour obtenir le temps restant d'une clé en secondes (PTTL en millisecondes)
- Avoir un système de messagerie *publishers/subscribers* avec un système de *channel*
 - Commande `SUBSCRIBE` pour s'inscrire à un canal
 - Commande `PUBLISH` pour envoyer un élément sur un canal
- Gérer des données géospatiales
 - <https://redis.io/docs/latest/develop/data-types/geospatial/>
- Gérer du JSON
 - <https://redis.io/docs/latest/develop/data-types/json/>

Autres possibilités

Et bien évidemment, il est possible d'interroger Redis avec son langage de programmation préféré en utilisant la bonne librairie :

- Java : *Jedis* ou *Lettuce*
(<https://redis.io/docs/latest/develop/connect/clients/java/>)
- PHP : Librairie *phpredis* (<https://github.com/phpredis/phpredis>)
- Node.js : Librairie *node-redis*
(<https://redis.io/docs/latest/develop/connect/clients/nodejs/>)
- Python : Librairie *redis-py*
(<https://redis.io/docs/latest/develop/connect/clients/python/>)

La persistance

Les données de Redis sont stockées en mémoire

Si on souhaite les conserver, il faut mettre en place la persistance

Deux possibilités qui peuvent se compléter :

- RDB (*Redis Database*) : Création de *snapshots* - Récupération complète des données
- AOF (*Append Only File*) : Log des opérations d'écritures reçues par le serveur - Vitesse et disponibilité

En créant des *snapshots* avec les commandes `SAVE` (synchrone) et `BGSAVE` (asynchrone)

- La commande `SAVE` bloquera les clients le temps de la sauvegarde
- Exemple : `SAVE 30 1000` effectuera une sauvegarde toutes les 30 secondes si au moins 1000 clés ont changé

La documentation est claire concernant l'utilisation de la persistance :

https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/

Bases orientées *documents*

Bases orientées documents

Une base de données orientée document est un programme informatique conçu pour stocker, récupérer et gérer des informations orientées document, également appelé données semi-structurées

Les bases de données orientées document peuvent apparaître comme une sous-classe des bases clés-valeurs mais la différence réside dans la façon dont les données sont traitées

Dans une base clé-valeur, les données sont considérées comme étant opaques par rapport à la base de données, alors qu'un système orienté document repose sur la structure interne du document pour permettre une recherche selon celle-ci

Bases orientées documents

Les bases de données documents contrastent fortement avec les bases de données relationnelles:

- Les bases de données relationnelles stockent les données dans des tables distinctes définies par le programmeur et un même objet peut être réparti sur plusieurs tables
- Les bases de documents document stockent toutes les informations relatives à un objet donné dans un seul document, et chaque document peut varier par rapport aux autres.

Cela permet de mapper des objets dans la base de données en une tâche simple, en éliminant normalement tout ce qui ressemble à un mappage relationnel-objet

Cela rend les bases orientées document attrayantes pour la programmation d'applications Web, qui sont soumises à des changements permanents, et où la rapidité de déploiement est un problème important.

Bases orientées documents

La nature structurée des documents permet de mettre au sein de chacun d'entre eux toute l'information concernant une unité d'information (ex : un produit, ses caractéristiques, etc...)

Cette unité d'information peut ainsi être récupérée plus tard avec une requête, là où une base de données relationnelle aurait eu besoin de faire des jointures

Bases orientées documents

Concernant le stockage et la représentation de l'information, une base de données orientée documents dispose dans l'absolu de plusieurs possibilités, par exemple : JSON, XML, YAML, ...

```
{  
  "titre": "Dark Matter",  
  "artiste": "Pearl Jam",  
  "date": {  
    "jour": 19,  
    "mois": 4,  
    "annee": 2024  
  },  
  "duree": 48,  
  "pistes": [  
    "Scared Of Fear",  
    "React, Repond",  
    "Wreckage",  
    ...  
  ]  
}
```

JSON

```
<album>  
  <titre>Dark Matter</titre>  
  <artiste>Pearl Jam</artiste>  
  <date>  
    <jour>19</jour>  
    <mois>4</mois>  
    <annee>2024</annee>  
  </date>  
  <duree>48</duree>  
  <pistes>  
    <titre>Scared Of Fear</titre>  
    <titre>React, Repond </titre>  
    <titre>Wreckage" </titre>  
    ...  
  </pistes>  
</album>
```

XML

Bases orientées documents

Le marché des bases de données orientées documents est très riche, avec de nombreux acteurs et solutions (gratuites, payantes, open-source, propriétaires, ...)

Certaines de ses solutions sont en mesure de prendre en charge plusieurs modèles de fonctionnement, outre le mode documents

Exemples :

- AllegroGraph (<https://allegrograph.com/>)
- ArangoDB (<https://arangodb.com/>)
- Couchbase (<https://www.couchbase.com/>)
- Amazon DocumentDB (<https://aws.amazon.com/fr/documentdb/>)
- MongoDB (<https://www.mongodb.com/fr-fr>)
- Oracle NoSQL Database (<https://www.oracle.com/fr/database/nosql/>)



MongoDB

UN EXEMPLE DE BASE ORIENTÉE DOCUMENTS

MongoDB

MongoDB est développé depuis 2007 par MongoDB Inc. Cette entreprise (initialement appelée 10gen) travaillait alors sur un système de type *platform-as-a-service*

En 2009, la même année que la sortie de la première version de MongoDB, l'entreprise migre vers un modèle open-source en offrant des services et du support liés à MongoDB

D'après DB-Engines (<https://db-engines.com/en/ranking>), MongoDB est aujourd'hui le 5ème SGBD le plus populaire et le 1er orienté documents et/ou NoSQL

Open-source, développé en C++, JavaScript et Python, 2 éditions existent : *Community* et *Enterprise*

- L'édition *Enterprise* offre des fonctionnalités supplémentaires

MongoDB permet de manipuler des objets structurés au format BSON (JSON binaire), sans schéma prédéterminé

- Ainsi, des documents peuvent être ajoutées à tout moment « à la volée », sans reconfiguration de la base

MongoDB

MongoDB prend en charge la réplication via un modèle maître-esclave à des fins de haute-disponibilité et le *sharding* de données à des fins de montée en charge

MongoDB bénéficie de nombreuses fonctionnalités de recherche de document (comparaison, logique, filtre de type, expression régulière, recherche texte, espace géographique, ...)

La base peut servir au stockage de fichiers de grande taille (GridFS) qui dépasse la taille maximale d'un BSON (16Mo)

Il est également possible de faire des requêtes d'agrégation sur de larges volumes de données

Toutes ces fonctionnalités peuvent être mises en place via une administration simple

MongoDB

Au sein de MongoDB, les données sont modélisées dans des objets JavaScript appelés *documents*

Les documents stockés par MongoDB sont de type BSON

- Les objets BSON sont la représentation binaire des objets JSON en leur ajoutant des améliorations et des capacités supplémentaires (expressions régulières, gestion des dates, etc.)

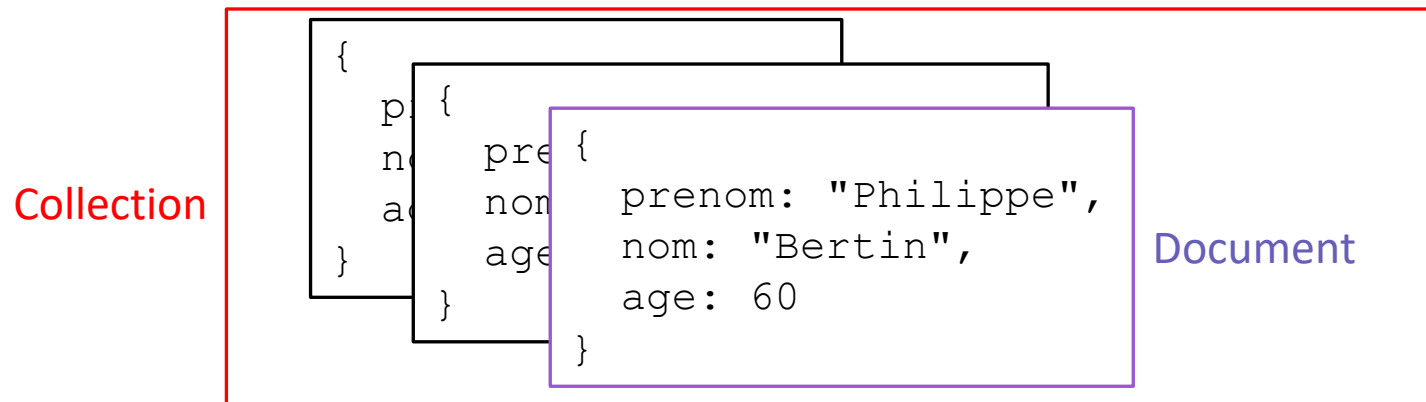
Chaque document possède un identifiant unique généré par MongoDB : la propriété `_id`

```
{
  _id: <ObjectID>,
  titre: "The Getaway",
  artiste: "Red Hot Chili Peppers"
  pistes: [
    { titre: "The Getaway", duree: 250 },
    { titre: "Dark Necessities", duree: 302 },
    ...
  ]
}
```

MongoDB

Les documents sont stockés dans des collections (qu'on pourrait rapprocher aux tables - mais l'analogie s'arrête là !)

Il faut voir une collection comme un amas de documents pas forcément uniformes. Car un document n'est jamais figé : les couples clé / valeur peuvent changer en fonction des données.



Ainsi, une base de données sous MongoDB contient des collections de documents

Un SGBD MongoDB est composé d'une ou plusieurs bases de données

MongoDB

SGBDR MySQL	SGBD MongoDB
Les bases de données ont des tables	Les bases de données ont des collections
Les tables ont des lignes	Les collections ont des documents
Les lignes ont des cellules/colonnes	Les documents ont des champs
Les champs contiennent des types de données simples	Les champs contiennent : des types de données simples, des tableaux ou d'autre documents
Les schémas sont rigides	Les schémas sont fluides

En pratique / Serveur

Pour utiliser MongoDB, on dispose de 2 possibilités :

- L'installer sur une machine
 - Il faut alors télécharger MongoDB, l'installer puis le configurer
 - Il existe une image Docker officielle si on souhaite mettre en place une instance MongoDB sur un serveur Docker
- Utiliser MongoDB en mode SaaS grâce à MongoDB Atlas (<https://www.mongodb.com/fr-fr/atlas>)
 - Il suffit de créer un compte sur MongoDB Atlas pour créer une base de données sur le cloud
 - C'est de loin la solution la plus simple pour tester MongoDB
 - Le niveau gratuit permet de créer une base de 512Mo maximum
 - Pour aller plus loin en termes de stockage ou performance, il convient alors de payer une souscription

En pratique / Client

De nombreux langages de programmation disposent de bibliothèques qui permettent de faire communiquer son programme avec MongoDB

Quand on fait du développement ou de l'administration, on a besoin d'un outil, de préférence graphique, qui puisse nous permettre d'administrer ou de manipuler la base de données

Pour MongoDB, on dispose :

- De l'outil en ligne de commande MongoDB Shell
- De l'outil graphique officiel et gratuit MongoDB Compass (<https://www.mongodb.com/products/tools/compass>)
- D'extensions pour les divers IDE, par exemple MongoDB for VS Code (<https://www.mongodb.com/products/tools/vs-code>)

JSON / BSON

Qu'apporte l'utilisation du BSON ? Des types supplémentaires par rapport au JSON

Types du JSON : *String, Number, Boolean, Object, Array, Null*

- On remarquera que le JSON ne propose de types précis pour les nombres (entier, flottant,...), ni de type pour les dates
- Quand on utilise le JSON, c'est aux développeurs de se mettre d'accord sur le format

Le BSON (format introduit par MongoDB en 2009) est le format utilisé par MongoDB pour stocker les documents

- Le BSON n'est pas lisible par un être humain, mais il peut être affiché en JSON pour devenir lisible

Les améliorations du BSON :

- Les documents BSON occupent moins d'espace et permettent une communication et un traitement plus rapides
- Dans chaque document, le BSON stocke certaines métadonnées comme la longueur des champs
- Les tableaux sont ordonnés et accessibles via l'index (position)
- Il fournit des types supplémentaires comme par exemple les dates, les entiers, les flottants, les tableaux d'octets

ObjectId

Tout document MongoDB doit contenir un `_id` qui contient une valeur unique

- Ce champs agit un peu comme une clé primaire pour le document et sa valeur doit être unique au sein d'une collection

Quand on insère un document dans un champ `_id`, MongoDB génère automatiquement un `_id` unique

- Quand le champ `_id` est généré automatiquement, sa valeur est générée en utilisant `ObjectId`
- `ObjectId` génère une valeur unique de 12 octets
 - Les 4 premiers octets représentent le *timestamp*
 - Les octets 5 à 9 sont une valeur aléatoire
 - Les 3 derniers octets constituent un compteur incrémenté

Exemple de valeur

- `ObjectId("9a121ff44cd41ab41799214896")`

Quelques règles

Un document ne peut pas avoir une taille supérieure à 16Mo

Un document BSON dans MongoDB supporte jusqu'à une profondeur de 100 niveaux pour les sous-documents

Quelques règles sur le nom des champs :

- Pas de caractère *null*
- Seuls les champs dans un tableau ou un sous-document peuvent commencer par un caractère \$
 - Dans tous les cas cette pratique n'est pas recommandée
- Il n'est pas possible d'avoir deux champs qui portent le même nom

Rechercher des documents

Globalement une recherche MongoDB respectera un même modèle de syntaxe

Méthode `find`

```
db.movies.find( { "title": "One Week" } )
```

Nom de la collection

Critère de la recherche

Ici, on recherche les films qui répondent au titre de *One Week*

Avant de faire une recherche, il faut se positionner sur la base de données concernée avec l'instruction `use`

```
use sample_mflix
```

Rechercher des documents

```
db.movies.find()
```

- Retourne tous les documents d'une collection ou d'une recherche

On peut indiquer des critères de recherche dans la méthode `find`

- `db.movies.find( { "title": "One Week" } )`
- Dans certaines situations, l'appel à `pretty()` permet un affichage plus propre
 - `db.movies.find( { "title": "One Week" } ).pretty()`
- Les résultats sont stockés dans un curseur qu'il est possible de parcourir
 - `var resultats = db.movies.find()`
 - `resultats.next()` pour avoir le résultat suivant
 - `resultats.hasNext()` pour savoir s'il y a un résultat ensuite

```
db.movies.findOne()
```

- Retourne le premier document trouvé, avec toujours la possibilité d'indiquer des critères de recherche

Recherche - Projection

Par défaut, une recherche retourne tous les champs

Si on veut uniquement des champs spécifiques, on l'indique dans la recherche

- `db.movies.find( { "title": "One Week" }, { "title": 1, "year": 1 } )`

Par défaut, `_id` est toujours affiché

- Pour le masquer, il faut l'indiquer explicitement
- `db.movies.find( { "title": "One Week" }, { "title": 1, "year": 1, "_id": 0 } )`

Recherche - Distinct & Count

On peut obtenir les valeurs distinctes d'un champ avec la fonction `distinct()`

- `db.movies.distinct("rated")`
- Avec un critère de recherche
 - `db.movies.distinct("rated", { "year": 2010 })`

Pour compter les documents, on dispose de la fonction `countDocuments()`

- `db.movies.countDocuments()`
- Avec un critère de recherche
 - `db.movies.countDocuments({ "year": 2010 })`
- La méthode `estimatedDocumentCount()` retourne une estimation du nombre de documents dans une collection mais n'accepte pas de critère de recherche(plus rapide mais moins précis)
 - `db.movies.estimatedDocumentCount()`

Recherche - Opérateurs

Pour effectuer des recherches plus avancées, on dispose d'opérateurs

Exemple avec l'opérateur `$eq`

- Teste l'égalité
- `db.movies.find({"year": 2010})` équivaut à
- `db.movies.find({"year": {$eq: 2010}})`

Opérateur	Utilité	Exemple
<code>\$eq</code>	Egal à	<code>{"year": {\$eq: 2010}}</code>
<code>\$ne</code>	Différent de	<code>{"year": {\$ne: 2010}}</code>
<code>\$gt</code> et <code>\$gte</code>	Supérieur à et supérieur ou égal à	<code>{"year": {\$gt: 2010}}</code>
<code>\$lt</code> et <code>\$lte</code>	Inférieur à et inférieur ou égal à	<code>{"year": {\$lte: 2010}}</code>
<code>\$in</code> et <code>\$nin</code>	Dans un ensemble de valeurs et inversement	<code>{"year": {\$in: [1956, 1980]}}</code>

Recherche - Opérateurs logiques

On dispose aussi d'opérateurs logiques

Exemple avec l'opérateur `$and`

- `db.movies.find({"rated": "PG-13", "year": 2010})`
équivalent à
- `db.movies.find({$and:
 [{"year": 2010}, {"rated": "PG-13"}]
})`

Opérateur	Utilité
<code>\$and</code>	ET logique
<code>\$or</code>	OU logique
<code>\$nor</code>	NOR logique - Qui ne satisfait aucune condition donnée
<code>\$not</code>	NOT logique - Négation d'une expression donnée

Recherche - Regex

Les expressions régulières permettent de faire des recherches avancées

- On utilise pour cela l'opérateur `$regex`
 - `db.movies.find({"title": {$regex: "Black"}}, {"title": 1})` recherche les titres de films qui contiennent la chaîne `Black`
- Le caractère `^` permet d'indiquer le début d'une chaîne et `$` la fin
 - `db.movies.find({"title": {$regex: "^Black"}}, {"title": 1})` recherche les titres de films qui commencent par la chaîne `Black`
 - `db.movies.find({"title": {$regex: "Black$"}}, {"title": 1})` recherche les titres de films qui terminent par la chaîne `Black`
- Pour faire une recherche insensible à la casse, on peut l'indiquer en option avec l'opérateur `$options`
 - `db.movies.find({"title": {$regex: "Black$", $options: "i"}}, {"title": 1})`
- On peut aller plus vite en utilisant la syntaxe alternative avec `/`
 - `db.movies.find({"title": /^Black/}, {"title": 1})`

Voir une documentation sur les expressions régulières pour avoir un aperçu complet

Recherche - Tableaux

Pour rechercher dans un tableau on procède comme habituellement

- `db.movies.find({"cast": "Julia Roberts"}, {"title": 1, "cast": 1})`
 - On obtient les films qui ont Julia Roberts dans le tableau des acteurs
- `db.movies.find({$and: [{"cast": "Julia Roberts"}, {"cast": "Hugh Grant"}]})`
 - On obtient les films qui ont Julia Roberts et Hugh Grant dans le tableau des acteurs

L'opérateur `$all` permet de simplifier cela pour une syntaxe équivalente

- `db.movies.find({"cast": {"$all": ["Julia Roberts", "Hugh Grant"]}})`

Recherche - Tableaux

Dans certain cas, on peut faire une recherche en mode tableau, mais l'ordre et le nombre d'éléments doit être respecté

- `db.movies.find({"languages": ["French", "English"]})`

Pour la projection

- `db.movies.find({"languages": "French"}, {"languages": 1})` affiche le tableau de langues de chaque film
- `db.movies.find({"languages": "French"}, {"languages.$": 1})` affiche juste les éléments qui correspondent dans le tableau

Recherche - Objets

Dans un document, il peut y avoir des objets imbriqués

Pour rechercher des documents avec des critères de recherche sur les objets imbriqués, MongoDB propose plusieurs possibilités

La méthode la plus simple est de rechercher directement l'objet

- `db.movies.find({"awards": { "wins": 1, "nominations": 0, "text": "1 win."}})`
- Ici la recherche est effectuée de façon exacte sur l'objet

Parfois, on a besoin de faire une recherche sur un seul critère de l'objet imbriqué

- `db.movies.find({"awards.wins": 3})`
- On utilise une syntaxe classique en utilisant le *dot* (.)
- On peut continuer à utiliser les opérateurs classiques (`$gt`, `$gte`, `$lt`, ...)

Recherches - Limite et Saut

Limiter le nombre de documents retournés

- On utilise la fonction `limit(x)`
- `x` permet d'indiquer le nombre de résultats souhaités
- La requête sera exécutée mais on n'obtiendra que les `x` premiers résultats
- Exemple :
 - `db.movies.find({"cast": "Julia Roberts"}).limit(3)`
- Une valeur à 0 ne fixe au final pas de limite

Sauter un certain nombre de documents

- On utilise la fonction `skip(x)`
- `x` permet d'indiquer le nombre de résultats à sauter
- Exemple :
 - `db.movies.find({"cast": "Julia Roberts"}).limit(3).skip(2)`

L'utilisation des deux mécanismes est couramment utilisé pour faire de la pagination

Recherches - Tri

Pour trier les documents dans le jeu de résultat, on utilise la fonction `sort()`

```
◦ db.movies.find(  
    {"cast": "Julia Roberts"},  
    {"title": 1, "_id": 0}  
)  
    .sort({"title": 1})
```

On indique le critère de tri en indiquant une valeur :

- 1 : tri par ordre croissant
- -1 : tri par ordre décroissant

On peut combiner les critères de tri

```
◦ db.movies.find({}, {"title": 1, "year": 1, "_id": 0})  
    .limit(50)  
    .sort({"year": 1, "title": -1})
```

Insertion

Pour ajouter un nouveau document dans une collection on utilise la fonction `insertOne()`

- On indique à la fonction le document à ajouter
- Exemple
 - `db.acteurs.insertOne({"name": "Will Smith", "year": 1968})`
- Une valeur `_id` est alors automatiquement générée
- Si la collection n'existe pas, elle est automatiquement créée

Si on a plusieurs documents à ajouter, on utilise la fonction `insertMany()` et on met tous les documents dans un tableau

- Exemple :
 - ```
db.acteurs.insertMany([
 {"name": "George Clooney", "year": 1961},
 {"name": "Julia Roberts", "year": 1967}
])
```

On peut indiquer une valeur pour `_id`, mais il est souvent préférable de laisser le moteur générer la valeur pour nous

- Deux valeurs `_id` identiques dans une même collection sera refusé

# Suppression

---

La fonction `deleteOne()` permet de supprimer un seul document dans une collection

- En paramètre, on indique un document qui représente une requête
- Si plusieurs documents correspondent aux critères de recherche, seul le premier document sera supprimé
- Exemples :
  - `db.acteurs.deleteOne({"name": "Julia Roberts"})`
  - `db.acteurs.deleteOne({"_id": ObjectId("66d4c9e90b6fd5be0a832446")})`

# Suppression

---

La fonction `deleteMany()` permet quant à elle de supprimer plusieurs documents

- En paramètre, on indique un document qui représente une requête
- Exemple : `db.actors.deleteMany({"year": 1967})`
- Dans le résultat on obtient le nombre de documents supprimés
- Pour supprimer tout le contenu d'une collection :
  - `db.actors.deleteMany({})`

MongoDB propose une autre fonction de suppression

- `findOneAndDelete()` : Retourne le premier document trouvé qui correspond aux critères de recherche et le supprime

# Remplacement

---

Pour remplacer un document par un autre, on dispose de la fonction `replaceOne()` qui prend en paramètre des critères de recherche et le document de remplacement

Exemple :

- `db.acteurs.replaceOne(`  
    `{ "_id": ObjectId("66d569e5f46004e63cd67053") },`  
    `{ "name": "Brad Pitt", "year": 1963 })`

Ne pas oublier qu'on remplace un document par un autre

- On ne modifie pas les champs avec `replaceOne()`

On n'indique pas de valeur pour `_id` dans le document de remplacement

- `_id` est immuable. On ne peut pas le modifier et il garde du coup la même valeur

# Remplacement / Création

---

La fonction `replaceOne()` peut accepter en troisième paramètre des options qui viendront modifier son comportement

L'option `{ upsert: true }` permet d'indiquer à MongoDB de créer un nouveau document si les critères de recherche du document à remplacer ne donnent rien

Exemple :

- ```
db.acteurs.replaceOne(  
  {"name": "Vin Diesel" },  
  {"name": "Sean Penn", "year": 1960},  
  { upsert: true })
```
- S'il n'y a pas d'acteur répondant au nom de *Vin Diesel*, alors un nouveau document pour *Sean Penn* sera créé

Remplacement

MongoDB propose une autre fonction de remplacement

- `findOneAndReplace()` qui permet de remplacer un document mais aussi de retourner un document
- Par défaut, la méthode retourne le document original

En troisième paramètre, on peut indiquer des options qui viendront modifier le comportement de la fonction

- Avec l'option `{ returnNewDocument: true }`, on peut lui dire de retourner le nouveau document
- L'option `sort` permet d'indiquer des critères de tri qui viendront influencer le premier document trouvé
- L'option `projection` permet d'indiquer les champs qu'on souhaite avoir dans le document retourné

Modification

Les fonctions de remplacement sont utiles mais si on souhaite modifier un seul champ dans un document, il convient de recopier l'intégralité du document, ce qui n'est pas du tout pratique

Pour faire cela on dispose de la fonction `updateOne()`

- Premier paramètre : un critère de recherche
- Deuxième paramètre : le ou les champs à modifier
- On utilisera dans le deuxième paramètre l'opérateur `$set`
- La fonction viendra alors modifier les champs indiqués dans le premier document trouvé
- Exemple :
 - `db.acteurs.updateOne(`
 `{"name": "Sean Connery"}, {$set: {"dead": true}})`
 - Ici, le champ `dead` est mis à `true`, ou bien il est créé s'il n'existe pas

Modification

La fonction `updateOne()` peut accepter en troisième paramètre des options qui viendront modifier son comportement

- L'option `{ upsert: true }` permet d'indiquer à MongoDB de créer un nouveau document si les critères de ne donnent rien

MongoDB propose une autre fonction de modification

- `findOneAndUpdate()` qui permet de modifier un document mais aussi de retourner un document
- Par défaut, la méthode retourne le document original

En troisième paramètre, on peut indiquer des options qui viendront modifier le comportement de la fonction

- Avec l'option `{ returnNewDocument: true }`, on peut lui dire de retourner le nouveau document
- L'option `sort` permet d'indiquer des critères de tri qui viendront influencer le premier document trouvé
- L'option `projection` permet d'indiquer les champs qu'on souhaite avoir dans le document retourné

Modification

Pour modifier en masse des documents, on dispose de la fonction `updateMany()`

La fonction fonctionne de la même manière

- Le premier paramètre permet d'indiquer les critères de recherche sur les documents à remplacer
- Le deuxième paramètre permet de modifier les valeurs des champs avec l'opérateur `$set`

Modification - Opérateurs

MongoDB propose d'autres opérateurs de modification que `$set`

- Dans la fonction utilisée, on peut en utiliser plusieurs

Opérateur	Utilité	Exemple
<code>\$set</code>	Modifie une valeur par une autre	<code>{ \$set: { "year": 1978 } }</code>
<code>\$inc</code>	Incrémente un champ numérique d'une certaine valeur (positive ou négative)	<code>{ \$inc: { "year": 2 } }</code>
<code>\$mul</code>	Multiplie un champ numérique	<code>{ \$mul: { "score": 2 } }</code>
<code>\$rename</code>	Renomme un champ	<code>{ \$rename: { "year": "birth_year" } }</code>
<code>\$currentDate</code>	Donne à un champ la date courante ou <i>timestamp</i>	<code>{ \$currentDate: { "created": true } }</code>
<code>\$unset</code>	Enlève un champ d'un document	<code>{ \$unset: { "dead": "" } }</code>
<code>\$setOnInsert</code>	Règle le champ dans le cas d'un <i>upsert</i>	<code>{ \$setOnInsert: { "created": new Date() } }</code>

Modification - Pipeline

L'utilisation d'un pipeline lors d'une modification permet de découper celle-ci en plusieurs étapes afin d'arriver à une modification finale

On met chaque étape de la modification dans un tableau

La syntaxe est la suivante :

- `db.collection.updateMany(
 <conditions>,
 [<mise à jour 1>, <mise à jour 2>, <mise à jour 3>,...],
 <options>
)`

Modification - Pipeline

Imaginons une collection acteurs qui contient un champs name

Voici une modification sous la forme d'un pipeline

```
◦ db.acteurs.updateMany({}, [
  { $set: { "tab_name": { $split: ["$name", " "] } } },
  { $set: {
    "prenom": { "$arrayElemAt": ["$tab_name", 0] },
    "nom": { "$arrayElemAt": ["$tab_name", 1] }
  } },
  { $project: {
    "prenom": 1,
    "nom": 1,
    "nom_complet": { $concat:
      [ { $toUpper: "$nom" }, " ", "$prenom" ]
    }
  } }
])
```

On a ici un pipeline de modification en 3 étapes :

- Le premier `$set` éclate le nom de l'acteur au niveau de l'espace en un tableau
- Le deuxième `$set` récupère le prénom et le nom dans le tableau
- Le `$project` effectue une projection pour récupérer le prénom, le nom et le nom complet avec le nom en majuscule

L'agrégation

Parfois, on a besoin d'effectuer des requêtes plus avancée, notamment de l'agrégation

- C'est le but de ce qu'on appelle le pipeline d'agrégation (*aggregation pipeline*)

L'idée est d'élaborer un pipeline qui va traiter les résultats

- Ce pipeline se compose de plusieurs étapes, chaque étape venant extraire de la donnée
- A la fin du pipeline, on obtient un résultat final

On implémente donc chaque étape sous la forme d'un tableau et on communique ce tableau à la méthode `aggregate()`

Ce fonctionnement permet d'élaborer des requêtes complexes de façon assez simple

L'agrégation

Voici un exemple de pipeline qui permet de sélectionner les films de 2010 et après puis de les regrouper année en comptant le nombre de films

```
[
  { $match: { year: { $gte: 2010 } } },
  { $group: { _id: "$year", nombre: { $sum: 1 } } }
]
```

On peut alors lancer la requête avec `aggregate()`

```
db.movies.aggregate([
  { $match: { year: { $gte: 2010 } } },
  { $group: { _id: "$year", nombre: { $sum: 1 } } }
])
```

L'agrégation

Voici quelques opérateurs qui peuvent être utilisés

- `$match` : Filtre les documents à prendre en compte pour la prochaine étape grâce à des critères de recherche
- `$group` : Regroupe les documents selon un critère et applique une expression d'accumulation
- `$project` : Effectue une projection pour garder que les champs qui intéressent
- `$sort` : Effectue un tri des documents
- `$limit` : Limite le nombre de résultats à envoyer à la prochaine étape

Il existe plein d'autres opérateurs (plus ou moins avancés) à découvrir dans la documentation

- <https://www.mongodb.com/docs/manual/reference/operator/aggregation-pipeline/>

Bases orientées *colonnes*

Bases orientées colonnes

Une base orientée colonnes stocke les tables de données par colonne plutôt que par ligne

L'utilisation pratique d'une base "colonnes" et d'une base "lignes" diffère peu : elles peuvent utiliser des langages de requête traditionnels tels que SQL pour charger des données et effectuer des requêtes

Les bases de données en lignes et en colonnes peuvent devenir la colonne vertébrale d'un système pour fournir des données aux outils d'extraction, de transformation, de chargement (ETL) et de visualisation de données.

Bases orientées colonnes

Toutefois, en stockant les données dans des colonnes plutôt que dans des lignes, la base de données peut accéder plus précisément aux données dont elle a besoin pour répondre à une requête plutôt que d'analyser et de supprimer des données non souhaitées dans des lignes

Les performances des requêtes sont augmentées pour certaines charges de travail

Une base de données orientée colonne sérialise toutes les valeurs d'une colonne, puis celles de la colonne suivante, etc...

- On retrouve dans les grandes lignes le concept d'index dans une base orientée lignes

Bases orientées colonnes

Le fait qu'une base orientée colonnes soit plus efficace ou non dépend beaucoup de la tâche considérée :

- Les opérations pour extraire toutes les données d'un objet donné (= la ligne entière) sont plus lentes.
- Un système basé sur des lignes peut extraire la ligne dans une seule lecture de disque, alors que de nombreuses opérations sur disque pour collecter des données à partir de plusieurs colonnes sont requises à partir d'une base de données à colonnes.
- Dans une application rolodex, par exemple, il est beaucoup plus courant de collecter le nom et le prénom de plusieurs lignes afin de créer une liste de contacts, plutôt que de lire toutes les données d'une adresse unique.

Encore plus vrai pour l'écriture de données dans la base de données, en particulier si les données ont tendance à être "rares" avec de nombreuses colonnes facultatives.

- Pour cette raison, les magasins de colonnes ont démontré d'excellentes performances dans le monde réel, en dépit de nombreux inconvénients théoriques

Bases orientées colonnes

Le marché des bases de données orientées colonnes comporte plusieurs acteurs et solutions (gratuites, payantes, open-source, propriétaires, ...)

Exemples :

- Google Bigtable (<https://cloud.google.com/bigtable/>)
- Cassandra (<https://cassandra.apache.org/>) - Inspiré de Google Bigtable
- Hbase (<https://hbase.apache.org/>) - Inspiré de Google Bigtable
- ClickHouse (<https://clickhouse.com/>)
- Oracle Database 12c
- Microsoft SQL Server



Apache Cassandra

UN EXEMPLE DE BASE ORIENTÉE COLONNES

Apache Cassandra

Initialement développée par Facebook, cette solution a été libérée en open source en juillet 2008

- Depuis le projet est porté par la fondation Apache.

Cassandra est aujourd'hui le 11ème SGBD le plus populaire et le 1er orienté colonnes selon DB-engines (<https://db-engines.com/en/ranking>)

Conçu pour gérer des quantités massives de données sur un grand nombre de serveurs paramétrés en cluster, en assurant une haute disponibilité en éliminant les points individuels de défaillance

- On parle en l'occurrence de distribution de la donnée entre serveurs et de réplication

Il permet une répartition robuste sur plusieurs centres de données, avec une réplication asynchrone sans master et une faible latence pour les opérations de tous les clients

Apache Cassandra

Cassandra met l'accent sur la performance pour atteindre le plus fort débit sur le maximum de nœuds même si cela se fait au détriment d'une latence élevée sur les lectures et les écritures

Cassandra est basée sur une structuration en paires clé-valeur de type *eventually consistent*

Cassandra ne peut ni faire de jointures ni de sous-requêtes

- Même s'il gère des tables, il n'est pas relationnel

Cassandra privilégie la dénormalisation des données

Pour interroger Cassandra, on utilise un langage dédié appelé CQL (*Cassandra Query Language*) qui ressemble très fortement au SQL

Apache Cassandra

La structure en cluster permet d'avoir une approche décentralisée

- Un protocole P2P (*peer-to-peer*) est utilisé afin que chaque nœud du système puisse communiquer avec les autres
 - Cela assure qu'il n'y ait pas un point unique de défaillance
- Chaque nœud est responsable d'une certaine quantité de données et prend en charge les requêtes liées à ses données
- Cassandra utilise un hachage pour distribuer les lignes au sein du cluster selon la clé de la ligne

La structure en cluster permet d'avoir une approche tolérante aux fautes

- La donnée est automatiquement répliquée entre les nœuds
- La stratégie de réplication permet de déterminer quels nœuds stockent les réplicas

La structure en cluster permet d'assurer la cohérence des données

- Cassandra permet de spécifier une stratégie de cohérence : ONE, QUORUM ou ALL
 - Cela permet d'indiquer combien de réplicas doivent répondre avant de considérer une opération en succès
 - Une cohérence élevée permet de garantir une donnée fiable, mais introduit une latence dans le système

Apache Cassandra

Pour son stockage, Cassandra utilise 3 phases

- Le *CommitLog* : Contient le log de toutes les modifications effectuées sur le nœud
 - Toute donnée sera dans un premier temps écrite dans le *CommitLog* avant d'être écrite dans *Memtable*
 - Cela assure la fiabilité de la donnée en cas d'arrêt brutal
- *Memtables* : Structure en mémoire où Cassandra stocke son cache
 - En général il y a 1 *memtable* d'actif par table
 - Quand le *memtable* est plein, il est écrit dans un *SSTable*
- *SSTables* : Structure utilisée par Cassandra pour stocker la donnée sur le disque
 - Toutes les *SSTables* sont créées sur le disque depuis les *memtables* ou sont envoyées par les autres nœuds

Pour la lecture, Cassandra va d'abord vérifier dans les *Memtables* et ensuite va interroger les *SSTables*

Apache Cassandra

Pour quels usages utiliser Apache Cassandra ?

- Des données temporelles indexées selon le temps
 - Le partitionnement permet de faire une partition par période temporelle (1 heure, 1 jour,...)
- Des applications web évolutives
 - On peut ajouter des nœuds de façon horizontale pour absorber la charge
 - On peut paramétrer une réplication multi-datacenters pour améliorer la latence
- Du flux de données ou du flux d'évènements
 - Le mécanisme de stockage de Cassandra permet d'ingérer rapidement et efficacement du flux de données
 - Dès que de la donnée est ingérée, elle est disponible pour interrogation, ce qui permet de l'analyse en temps réel
- Des applications qui nécessitent une haute disponibilité
 - Il n'y a pas un seul point d'échec
 - On peut spécifier une stratégie de cohérence

Mise en place de Cassandra

Pour installer Cassandra, il est nécessaire d'avoir sur la machine concernée :

- Une JRE Java (Java 8 pour Cassandra 4, Java 11 pour Cassandra 5)
- Python 2.7+ ou Python 3.6+ pour utiliser `cqlsh` (*Cassandra Query Language Shell*) - L'outil en ligne de commande pour interagir avec Cassandra

Plusieurs possibilités d'installation :

- Depuis un fichier `tar` téléchargé depuis le site de Cassandra
- En utilisant un gestionnaire de paquet (`apt`, `yum`,...)
- En utilisant une image Docker (le plus simple)
- En utilisant une VM déjà paramétrée chez un *provider* de *cloud*

Mise en place de Cassandra

Pour installer depuis un fichier `tar`

- Décompresser et désarchiver le fichier `tar.gz` avec `tar -xzvf <fichier>`
- Aller dans le répertoire créé

Pour installer depuis un gestionnaire de paquets

- Ajouter le *repository* Apache Cassandra dans son gestionnaire de paquets
- Ajouter les clés du *repository*
- Faire un *update* et un *install* du paquet

Pour lancer une instance avec Docker

- On utilise l'image officielle `cassandra` (`cassandra:latest` pour la dernière version) - https://hub.docker.com/_/cassandra

Dans tous les cas, on paramètre son serveur en modifiant le fichier `cassandra.yaml`

- On le trouve dans le répertoire `conf/` (fichier `tar`) ou dans `/etc/cassandra` (paquet ou Docker)

Cassandra avec Docker

Pour lancer une instance avec Docker

- `docker run --name cassandra1 --network cassnet -d cassandra:latest`

Pour se connecter et utiliser cqlsh

- `docker run -it --network cassnet --rm cassandra:latest cqlsh cassandra1`

Pour ajouter un nouveau nœud au cluster

- `docker run --name cassandra2 -d --network cassnet -e CASSANDRA_SEEDS=cassandra1 cassandra:latest`

Pour persister la donnée, on pourra faire un *bind mount* ou utiliser un volume

- `docker run --name cassandra1 --network cassnet -v /mon/repertoire/donnees:/var/lib/cassandra -d cassandra:latest`

Utiliser Cassandra - CQL

On interagit avec Cassandra en utilisant le CQL

Basiquement, on dispose de 4 briques primordiales :

- *Keyspaces* : Il s'agit d'un espace de noms, comparable à une base de données en SQL
 - Une *keyspace* contiendra des tables et on y trouvera des paramétrages de réplication et des stratégies d'utilisation
- *Tables* : Similaires aux tables SQL
 - Les tables CQL contiennent des lignes
 - Cependant, de part la nature de Cassandra, les choix de clés primaires auront un rôle crucial lors de l'élaboration des tables
- *Columns* : Chaque table contient plusieurs colonnes, avec chacune un type de donnée
 - Avec Cassandra, les colonnes peuvent être dynamiques, permettant à certaines lignes d'avoir des colonnes que d'autres n'auraient pas
- *Primary Keys* : Contient la clé de partition (*partition key*) et optionnellement les colonnes de clusterisation (*clustering columns*)
 - La clé primaire détermine comment la donnée est distribuée à travers les nœuds
 - La clé de partition assure que la donnée est distribuée uniformément
 - Les colonnes de clusterisation déterminent l'ordre des données dans une partition

Utiliser Cassandra - CQL

Cassandra propose une variété de types :

- Des types basiques : Cela inclut les types habituels comme *int*, *text*, *float*, *boolean*
- Des types qui permettent de gérer des collections afin de stocker plusieurs valeurs dans une même colonne
 - *lists*
 - *sets*
 - *maps*
- Des types définis par l'utilisateur (UDT - *User-Defined Type*) en combinant plusieurs types basiques ou de collections
- Des tuples constitués d'un ensemble ordonné de composants de types différents

Utiliser Cassandra - CQL

Créer un *keyspace* - CREATE KEYSPACE

- `CREATE KEYSPACE demo WITH replication = { 'class': 'SimpleStrategy', 'replication_factor': '3' };`

Utiliser un *keyspace* - USE

- `USE demo;`

Créer une table - CREATE TABLE

- `CREATE TABLE utilisateurs (id UUID PRIMARY KEY, login TEXT, email TEXT);`

Modifier la structure d'une table - ALTER TABLE

- `ALTER TABLE utilisateurs ADD date_naissance DATE;`

Utiliser Cassandra - CQL

Insérer de la donnée - INSERT INTO

- `INSERT INTO utilisateurs (id, login, email) VALUES (uuid(), 'Franck', 'franck@example.com');`

Sélectionner de la donnée - SELECT

- `SELECT * FROM utilisateurs;`
- `SELECT * FROM utilisateurs WHERE id = 2c5ee6b9-aa14-445c-990f-48fcd517fe39;`

Mettre à jour de la donnée - UPDATE

- `UPDATE utilisateurs SET email = 'f@example.com' WHERE id = 2c5ee6b9-aa14-445c-990f-48fcd517fe39;`

Supprimer de la donnée - DELETE FROM

- `DELETE FROM utilisateurs WHERE id = 2c5ee6b9-aa14-445c-990f-48fcd517fe39;`

Vider une table - TRUNCATE

- `TRUNCATE utilisateurs;`

Utiliser Cassandra - CQL

Pour regrouper plusieurs opération dans une opération atomique, on utilise les *batches*

- On retrouve la même idée que les transactions en SQL
- `BEGIN BATCH` pour commencer un *batch*
- `APPLY BATCH` pour le terminer

```
BEGIN BATCH
INSERT INTO utilisateurs (id, login, email)
    VALUES (uuid(), 'Franck', 'franck@example.com');
INSERT INTO utilisateurs (id, login, email)
    VALUES (uuid(), 'Pauline', 'pauline@example.com');
INSERT INTO utilisateurs (id, login, email)
    VALUES (uuid(), 'Audrey', 'audrey@example.com');
APPLY BATCH;
```

Utiliser Cassandra - CQL

Décrire une structure - DESCRIBE

- `DESCRIBE keyspaces;` pour avoir la liste des *keyspaces*
- `DESCRIBE utilisateurs;` pour décrire la table `utilisateurs`

Obtenir la stratégie de cohérence utilisée

- `CONSISTENCY`

Modifier la stratégie de cohérence

- `CONSISTENCY QUORUM`

Exécuter un script CQL

- `SOURCE 'chemin/vers/le/fichier.cql'`

Modélisation des données

Lorsqu'on va utiliser Cassandra, en raison de sa nature de base de données orientée colonnes, il est nécessaire d'adopter une autre façon de pensée lorsqu'on va modéliser les tables

Dans une base de données relationnelle, on va raisonner sous la forme d'entités et de relations

Avec Cassandra, la modélisation va commencer en ayant à l'esprit les requêtes qu'on va souhaiter exécuter

La modélisation aura aussi un impact sur la distribution et la réplication de la donnée au sein du cluster

On fera appel à la dénormalisation

- La dénormalisation engendre de la redondance qui sera nécessaire avec Cassandra vu que les jointures n'existent pas

Modélisation des données

Lors de la modélisation la clé de partition sera capitale afin d'assurer une distribution de la donnée équilibrée

La clé primaire est la combinaison de :

- La clé de partition
- D'éventuelles colonnes de clusterisation

L'ordre est très important lors de la modélisation pour certaines requêtes

- Par exemple, si on veut avoir de la donnée entre deux dates, avoir une date comme colonne de clusterisation correctement triée permettra d'avoir une requête très efficace

Cassandra propose d'autres mécanismes

- Les vues matérialisées (*materialized views*)
- Les index secondaires (*secondary indexes*)

Modélisation des données

La stratégie de réplication

- `SimpleStrategy` : Pour les déploiements sur un simple datacenter
 - Avec un facteur de réplication de 3, Cassandra s'assurera que 3 copies de la donnée existe dans le cluster
- `NetworkTopologyStrategy` : Pour les déploiements répartis sur plusieurs datacenter
 - Cette stratégie permet de spécifier le facteur de réplication par datacenter

Le facteur de réplication

- Combien de réplikas d'une donnée soit exister dans le cluster

La stratégie de cohérence :

- ONE
- QUORUM
- ALL

Les types de données

Il est important de choisir le bon type pour chaque colonne pour des raisons de précision, de stockage, de performance et de fonctionnalités

Types basiques principaux

- Texte: TEXT
- Numérique : INT (32 bits), SMALLINT (16 bits), TINYINT (8 bits), FLOAT, DOUBLE, DECIMAL
- Temporel : DATE, TIME, TIMESTAMP
- Autre : BLOB, BOOLEAN

Types complexes principaux

- Collections : LIST (collection ordonnée), SET (collection non-ordonnée d'éléments uniques), MAP (paires clé/valeur)
 - ```
CREATE TABLE utilisateurs (
 id UUID PRIMARY KEY, login TEXT,
 emails SET<TEXT>
);
```
- Tuples : Pour grouper des éléments entre eux qui seront ordonnés
- Types définis par l'utilisateur (UDT) : Types personnalisés

# Clés primaires et colonnes de clusterisation

---

La clé primaire est un aspect essentiel de la modélisation

- Elle identifie une ligne dans une table et influence l'endroit et la manière dont la ligne est stockée

La clé primaire est la combinaison de :

- La clé de partition : déterminera quel nœud dans le cluster stockera la donnée
- De colonnes de clusterisation : détermineront l'ordre dans lequel les lignes concernées seront stockées dans le nœud

Une récupération de données efficace se base sur la clé de partition

- Si on fournit une clé de partition, la base de données n'aura pas à parcourir tous les nœuds pour trouver la donnée

# Clés primaires et colonnes de clusterisation

---

Les éventuelles colonnes de clusterisation déterminent l'ordre dans lequel les lignes concernées seront stockées dans le nœud

- Par défaut, au sein d'une partition les lignes sont stockées selon les valeurs de la colonne de clusterisation de façon croissante
- Quand il y a plusieurs colonnes de clusterisation, la première colonne est le premier déterminant avec un ordre croissant, puis la seconde, puis la troisième, etc...
- Ces colonnes permettent d'avoir des requêtes très efficaces quand on va rechercher sur un intervalle

Pour bien modéliser sa clé primaire, il faut bien connaître les requêtes envisagées

- L'ordre des colonnes de clusterisation permettra d'optimiser les requêtes

Bien choisir aussi sa clé primaire pour s'assurer d'une distribution uniforme

- On pourra utiliser un champ de type UUID en clé de partition pour s'assurer d'une distribution uniforme

# Création de table

---

## Une simple clé primaire avec une clé de partition

- `CREATE TABLE utilisateurs (  
    id UUID PRIMARY KEY, login TEXT, email TEXT);`

## Une clé primaire avec une clé de partition et des colonnes de clusterisation

- `CREATE TABLE commandes (  
    client_id UUID, date TIMESTAMP,  
    commande_id UUID, produit TEXT,  
    PRIMARY KEY (client_id, date, commande_id)  
);`
- Ici, toutes les commandes d'un même client seront stockées dans le même nœud et triées par date et par numéro de commande

# Création de table

---

```
CREATE TABLE utilisateurs (
 id UUID PRIMARY KEY, login TEXT, email TEXT);
```

- Ici, chaque identifiant d'utilisateur sera dans une unique partition dans le cluster, assurant la distribution de la donnée

```
CREATE TABLE utilisateurs (
 id UUID, date TIMESTAMP, message TEXT,
 PRIMARY KEY (id, date));
```

- **id est la clé de partition, date est la colonne de clusterisation**
  - Les messages d'un utilisateur seront stockés ensemble, triés par date

```
CREATE TABLE meteo (
 ville TEXT, date DATE,
 temperature DOUBLE, humidity DOUBLE,
 PRIMARY KEY((ville, date)));
```

- **La combinaison ville/date agit comme une clé de partition**
  - Les données météo d'une ville particulière pour une date donnée seront stockés dans la même partition

# Modification de tables

---

## Ajouter une colonne

- `ALTER TABLE utilisateurs ADD date_naissance DATE;`

## Supprimer une colonne

- `ALTER TABLE utilisateurs DROP date_naissance;`
- Quand une colonne est supprimée, Cassandra ne libère pas tout de suite l'espace. La donnée est marquée pour la suppression lors du prochain compactage (voir *compaction strategy*)

## Modifier le type d'une colonne

- `ALTER TABLE utilisateurs ALTER date_naissance TYPE TEXT;`

## Renommer une colonne

- `ALTER TABLE utilisateurs RENAME date_naissance TO date_n;`

## Il n'est pas possible de modifier une clé primaire

- Il faudra créer une nouvelle table à partir de l'ancienne pour cela

## Supprimer une table

- `DROP TABLE utilisateurs;`

# Le requêtage

---

On a déjà vu comment sélectionner de la donnée avec `SELECT` et `WHERE`

Si on sélectionne via la clé de partition, Cassandra saura où trouver la donnée

- `SELECT * FROM utilisateurs  
WHERE id = 2c5ee6b9-aa14-445c-990f-48fcd517fe39;`

Si on sélectionne avec un filtre qui n'est pas la clé de partition, Cassandra ne saura pas où trouver la donnée

- `SELECT * FROM utilisateurs  
WHERE login = 'florent';` sera en erreur
- Il faut alors forcer la recherche sur tous les nœuds, ce qui aura un impact sur les performances
- `SELECT * FROM utilisateurs  
WHERE login = 'florent' ALLOW FILTERING;`

# Le requêtage

---

Quand une table comporte une clé primaire composée (clé de partition et colonne(s) de clusterisation), il y a des aspects à considérer sur le WHERE

- La clé de partition doit être indiquée dans le WHERE
- Les colonnes de clusterisation peuvent être interrogées dans leur ordre de définition

## Utilisation du IN

- ```
SELECT * FROM utilisateurs  
WHERE id IN (2, 5, 8);
```

Création d'un index

- ```
CREATE INDEX ON utilisateurs (login);
```
- L'index permet de requêtage dans des colonnes qui ne font pas partie de la clé primaire mais nécessitent une mise à jour lors des modifications

# Le requêtage

---

Comme en SQL on retrouvera l'agrégation avec le `GROUP BY` et les différentes fonctions (`COUNT`, `SUM`, `AVG`, `MIN`,...)

- L'agrégation ne pourra se faire que dans la même partition

De même, l'agrégation peut être utilisée avec `HAVING`

